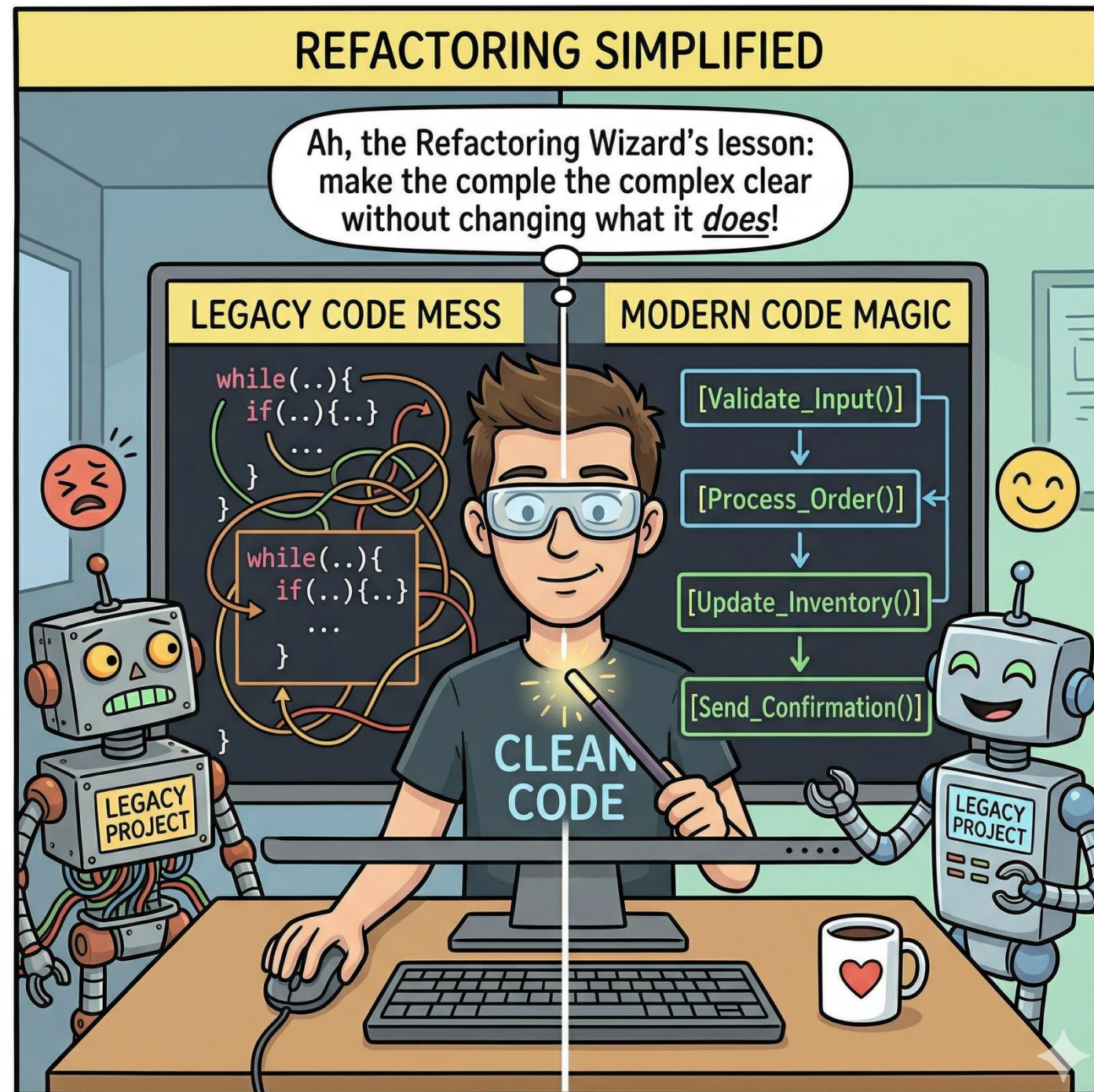


Refactoring

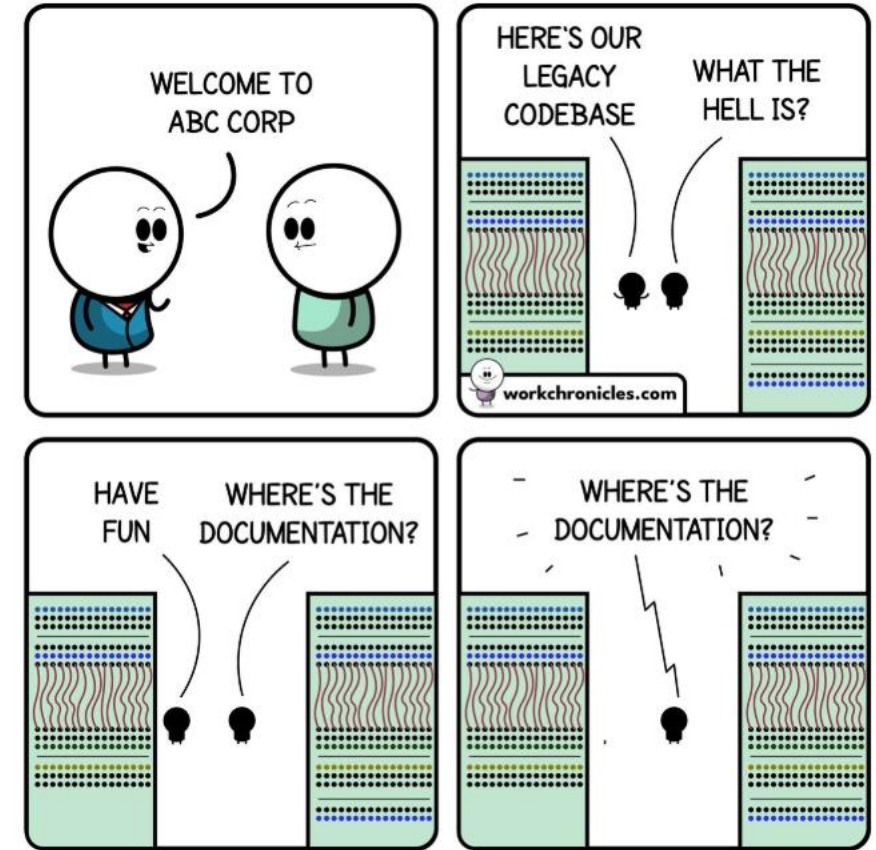
Design and Analysis of
Software Systems

Spring 2026



Lehman & Belady: Laws of Software Evolution (1974)

- **Continuing Change** – Systems must be continually adapted else they become progressively less satisfactory!
- **Increasing Complexity** – As a system evolves its complexity increases **unless work is done to maintain or reduce it.**
 - *Orkut?!*
 - *facebook -> meta*
 - *Twitter -> X, Koo?*

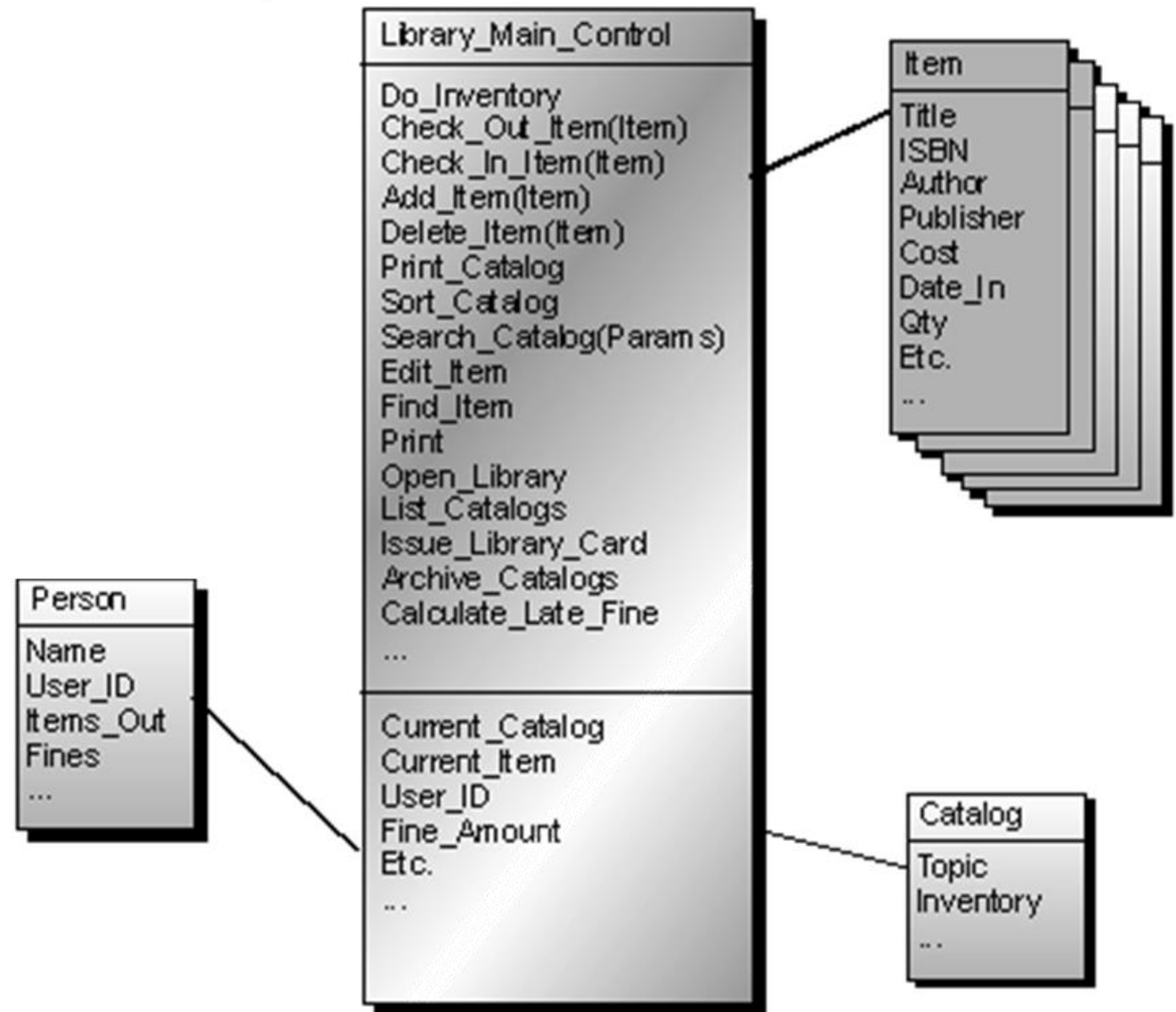


Comics about work. Made with ♥ & lots of coffee.
Get the comics straight to your Inbox. Join the Newsletter.

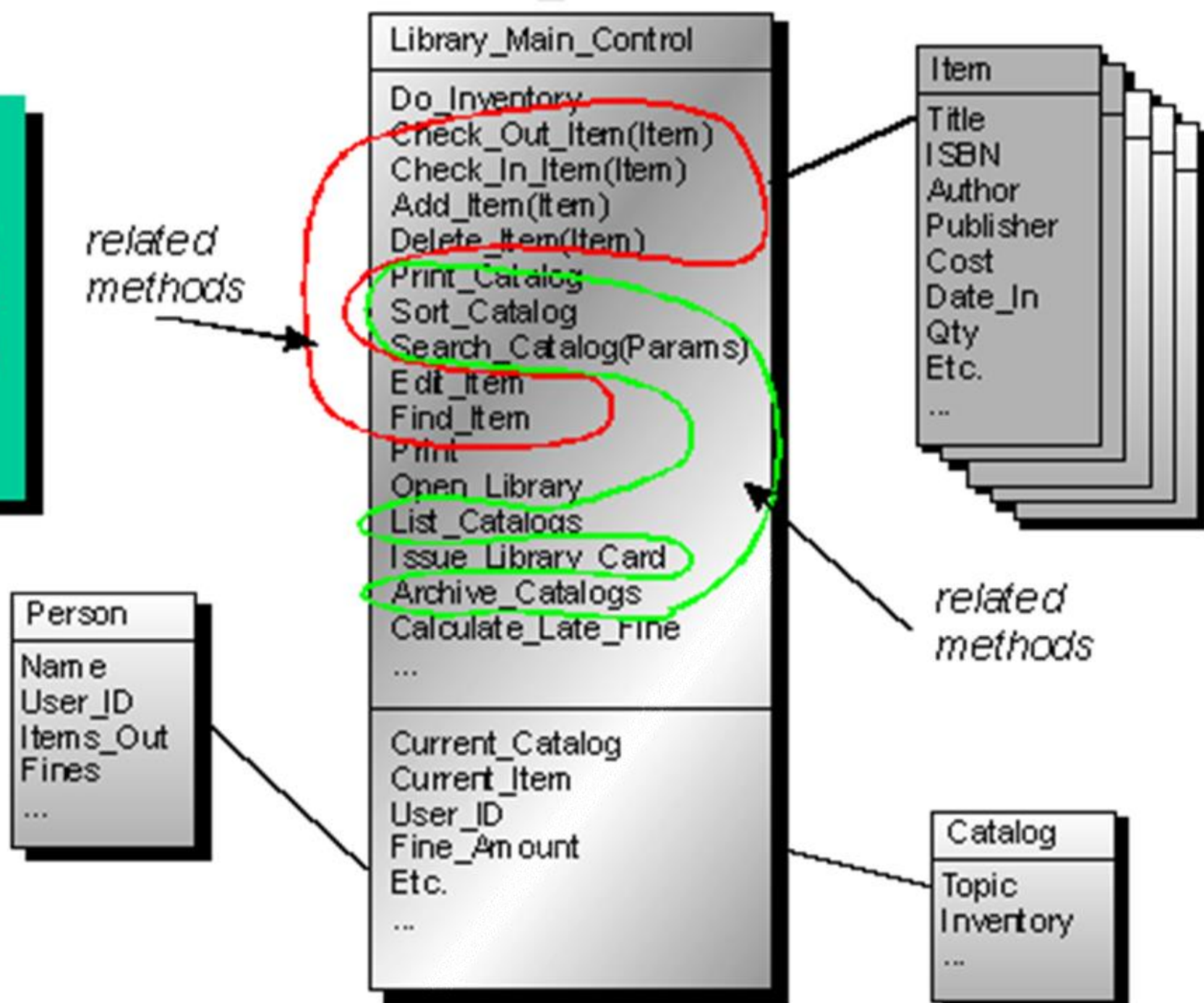
Work Chronicles
workchronicles.com

Library System – Existing Design

What areas do you see as potential problem areas?
Why did you identify each of those areas?



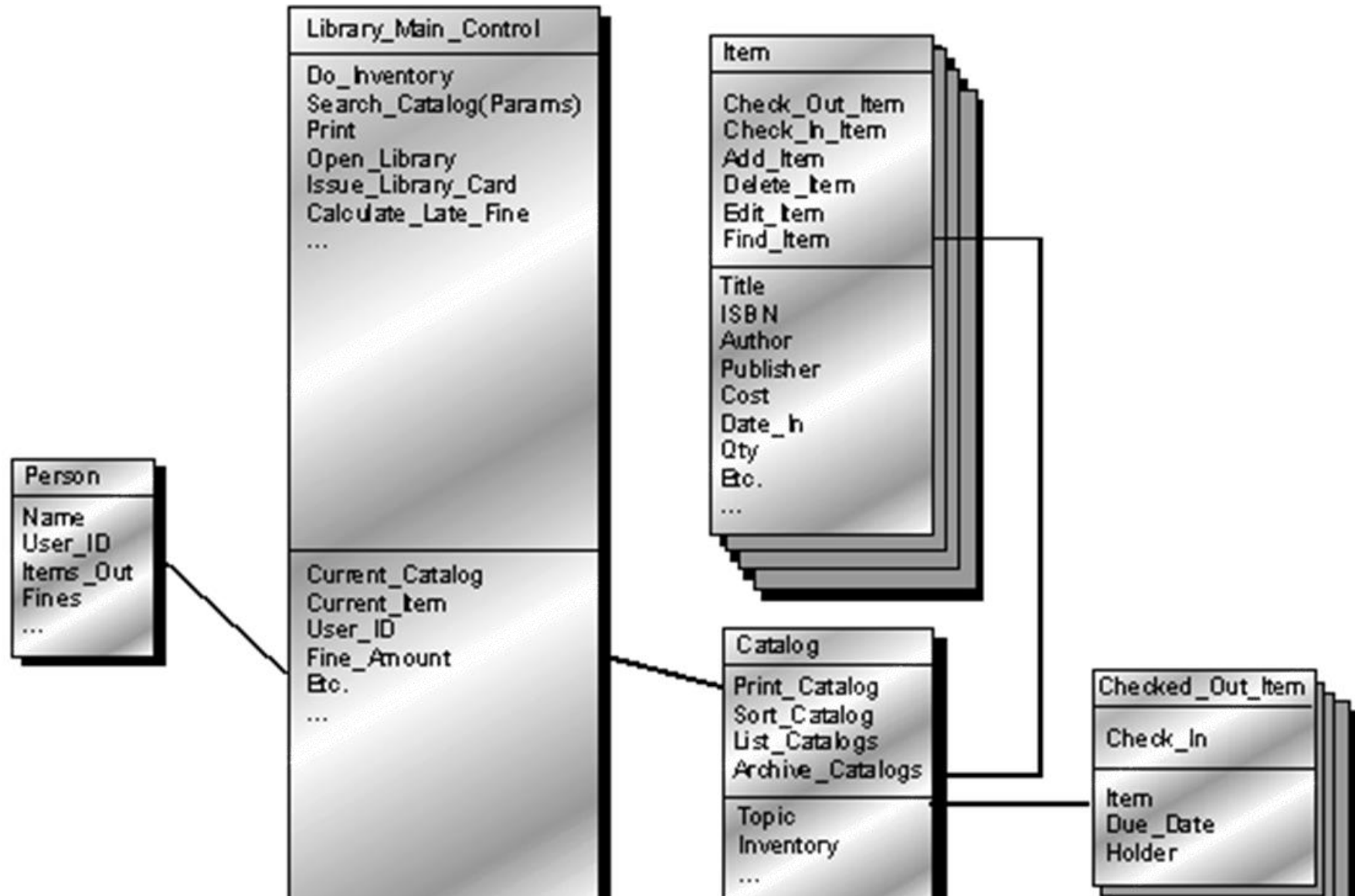
Step 1:
Identify or categorize
related attributes and
operations according
to contracts.



**Final Step:
Remove all transient
associations, replacing
them as appropriate
with type specifiers to
attributes and
operations arguments**



Library System – Changing the design



From this...



Source: Mike Lutz, RIT

To this...



Source: Mike Lutz, RIT

Design (Code) Refactoring!

- Improving code **without changing functionality**

```
let x = 10  
let y = 20  
let z = x + y  
console.log(z)
```

```
const firstNumber = 10  
const secondNumber = 20  
const total = firstNumber + secondNumber  
console.log(total)
```

Same functionality, Better readability, Easier maintenance

User Persona: Developers!

Design (UI/UX) Refactoring!

(Not our focus for today)

- Improving the visual output **without changing functionality** and **increase the user attention**

Too many options to understand – Cognitive Load

Difficulty in navigation flow – UX Debt

Complex control flow - Inclusive Design

User Persona: End Users!

Refactoring is moving from "it works" to "it's well-built."

Who Cares and Why?



Software / Component	Bug Type / Issue	Lifespan	Year Fixed ¹
Libpng	Graphics library flaw	30 Years	2026
OpenBSD	Integer overflow vulnerability	27 Years	2026
Mozilla Firefox	Tooltip UI bug	22 Years	2023
FFmpeg	Out-of-bounds write flaw	16 Years	2026
Unix Sudo Command	Privilege escalation vulnerability	~12 Years	2020 / 2021
Windows File Explorer	UI lag on dual-GPU laptops	10+ Years	2025

¹Source: <https://www.csoonline.com/article/570815/10-old-software-bugs-that-took-way-too-long-to-squash.html#:~:text=Here%20are%2014%20noteworthy%20bugs,Date%20fixed%3A%20February%202026> (Mar 5, 2026)

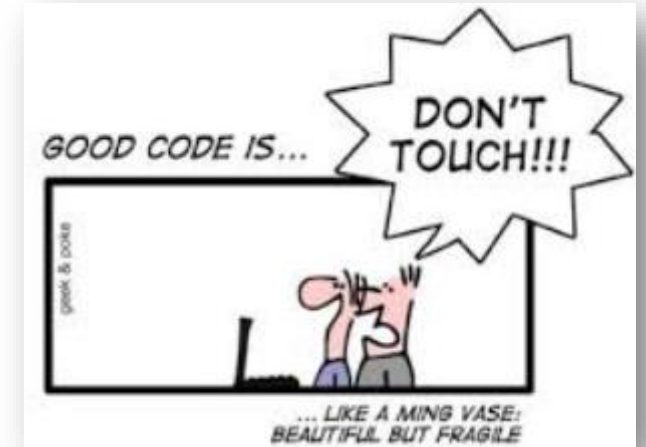
*Sora is killed! - <https://help.openai.com/en/articles/20001152-what-to-know-about-the-sora-discontinuation> (Apr 26, 2026)

What if? I do not refactor my code!

- Too Many Bugs (Fragile Code)
- Code is hard to understand – Low Readability
- Cost to Identify the Bug Rises (Engineer Time = Money)
- Slowed Development & Scaling Difficulties
- Growing Complexity & Technical Debt
- Missed Deadlines & Higher Long-Term Costs

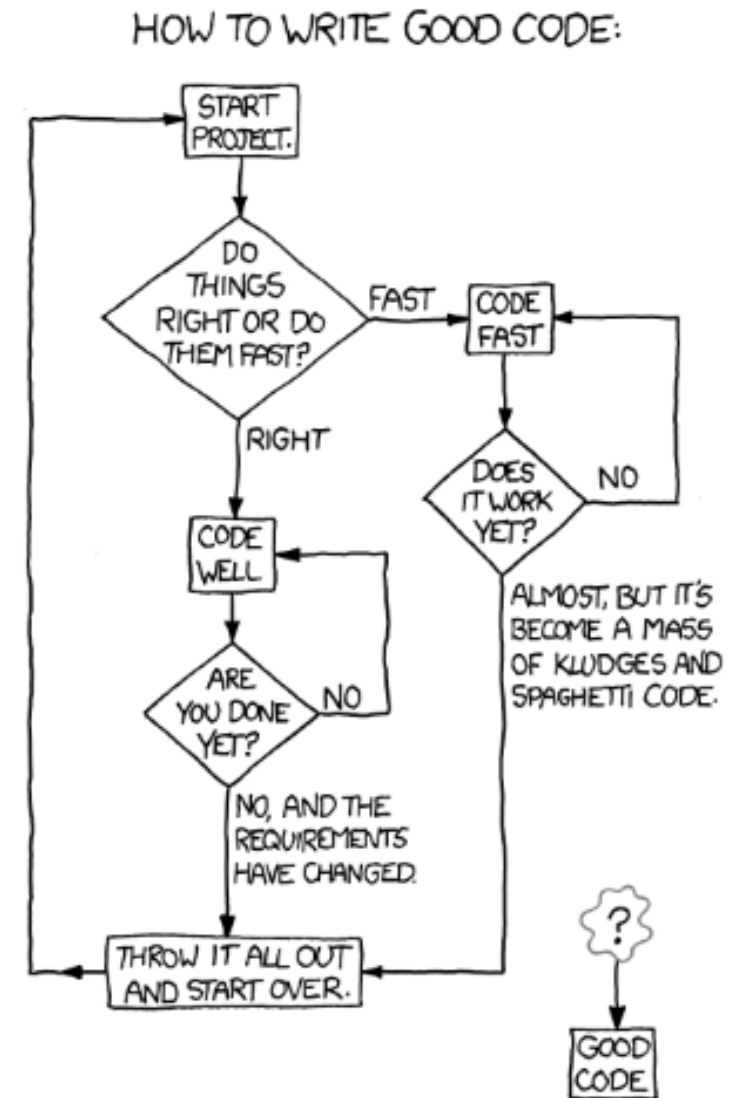
*Any fool can write code that a computer can understand.
Good Programmers write code that humans can
understand# – Martin Fowler*

#<https://refactoring.com/>



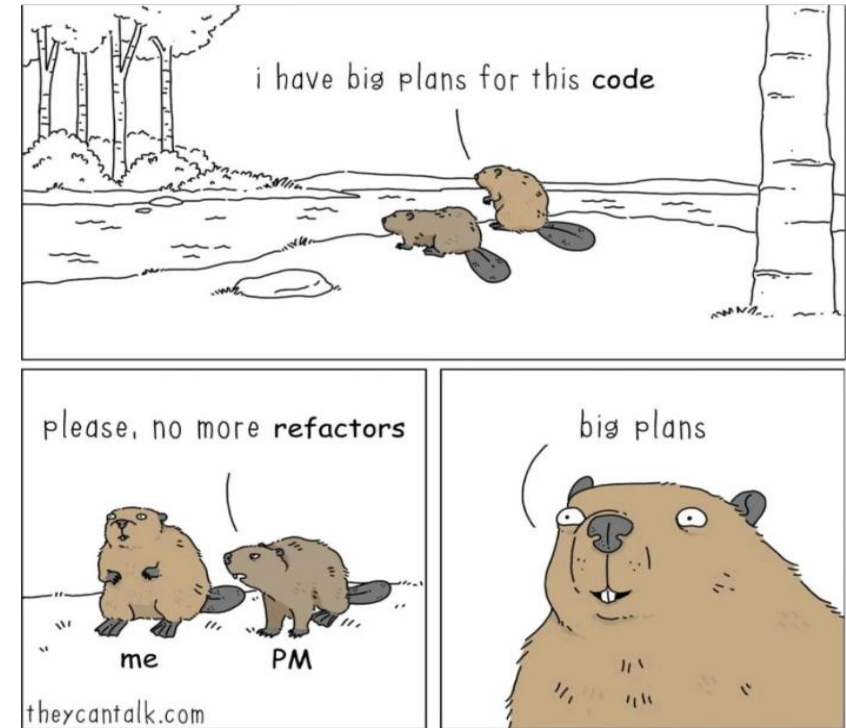
What causes Design Smells?

- Inappropriate use of patterns
- Language limitations
- Procedural thinking in OO
- Viscosity
 - Software viscosity
 - Environment viscosity
- Non-adherence to best practices and processes



Refactoring Principles!

- Do not refactor and extend a system at the same time
 - Make a clear separation between the two activities
- Have good tests in place before you begin refactoring
 - Run the tests often
 - Catch defects immediately
- Take small steps
 - Many localized changes result in a larger-scale change
 - Test after each small step



Code Smell – Long Functions!

```
def process_order(order):  
    total = 0  
    for item in order:  
        total += item["price"]  
  
    tax = total * 0.18  
    shipping = 50 if total < 500 else 0  
  
    final = total + tax + shipping  
    print("Total:", total)  
    print("Tax:", tax)  
    print("Shipping:", shipping)  
    print("Final:", final)
```

```
def calculate_total(order):  
    return sum(item["price"] for item in order)  
  
def calculate_tax(total):  
    return total * 0.18  
  
def calculate_shipping(total):  
    return 50 if total < 500 else 0  
  
def process_order(order):  
    total = calculate_total(order)  
    tax = calculate_tax(total)  
    shipping = calculate_shipping(total)  
  
    final = total + tax + shipping  
    print("Final:", final)
```

Mystery Names!

bad code

```
function d(a, b, c) {  
  // Check if valid  
  if (a && typeof a === 'number' && b && typeof b === 'number') {  
    let r = 0;  
    // Calculate square if even  
    if (a % 2 === 0) {  
      r = a * a;  
    } else {  
      // Calculate cube if odd  
      r = a * a * a;  
    }  
    // Multiply by b  
    r = r * b;  
    // Return result, c not used  
    return r;  
  }  
  return 0; // Invalid  
}
```

refactored

```
function isPositiveNumber(value) {  
  return value && typeof value === 'number';  
}  
  
function calculatePowerValue(number) {  
  if (number % 2 === 0) {  
    return number * number; // Even: square  
  }  
  return number * number * number; // Odd: cube  
}  
  
function processCalculation(inputValue, multiplier) {  
  if (!isPositiveNumber(inputValue) || !isPositiveNumber(multiplier)) {  
    return 0; // Invalid input handle early  
  }  
  return calculatePowerValue(inputValue) * multiplier;  
}
```


Duplicate Code!

bad code

```
function calculateUserDiscount(price){  
  let discount = price * 0.1  
  return price - discount  
}  
  
function calculateAdminDiscount(price){  
  let discount = price * 0.1  
  return price - discount  
}
```

refactored

```
function calculateDiscount(price, percentage){  
  return price - (price * percentage)  
}  
  
calculateDiscount(100, 0.1)
```

Poor Variable Names?

bad code

```
<button onclick="a()">Click</button>

<script>
function a(){
    let b = document.getElementById("price").value
    alert(b)
}
</script>
```

refactored

```
<button onclick="showPrice()">Show Price</button>

<script>
function showPrice(){
    const price = document.getElementById("price").value
    alert(price)
}
</script>
```

Complex Conditions

```
function getMembershipLevel(score) {  
  if (score > 1000) {  
    if (score < 5000) {  
      return "Bronze";  
    } else if (score < 10000) {  
      return "Silver";  
    } else {  
      return "Gold";  
    }  
  } else {  
    return "Explorer";  
  }  
}
```

bad code

refactored

```
function getMembershipLevel(score) {  
  if (score <= 1000) return "Explorer";  
  if (score < 5000) return "Bronze";  
  if (score < 10000) return "Silver";  
  return "Gold"; // Score is 10000 or more  
}
```

Technique – Extract Function

```
function checkout(cart){
  let total = 0
  cart.forEach(item => {
    total += item.price
  })

  let tax = total * 0.18
  let final = total + tax

  return final
}
```

bad code

refactored

```
function calculateTotal(cart){
  return cart.reduce((sum,item)=> sum + item.price ,0)
}

function calculateTax(total){
  return total * 0.18
}

function checkout(cart){
  const total = calculateTotal(cart)
  const tax = calculateTax(total)

  return total + tax
}
```


Technique – Replace Magic Numbers

bad code

```
if score > 85:  
    print("Excellent")
```

refactored

```
EXCELLENT_SCORE = 85  
  
if score > EXCELLENT_SCORE:  
    print("Excellent")
```

Technique – Simplify Conditionals

bad code

```
if(user.isActive == true){  
    return true  
}else{  
    return false  
}
```

refactored

```
return user.isActive
```

Code Smells Within Classes

- **Comments**
 - Are the comments necessary?
 - Do they explain "why" and not "what"?
 - Can you refactor the code so the comments aren't required?
 - Remember, you're writing comments for people, not machines.
- **Long Method**
 - Shorter method is easier to read, easier to understand, and easier to troubleshoot.
 - Refactor long methods into smaller methods if you can
- **Long Parameter List**
 - The more parameters a method has, the more complex it is.
 - Limit the number of parameters you need in a given method, or use an object to combine the parameters.
- **Duplicated code**
 - Stamp out duplication whenever possible.
 - [Don't Repeat Yourself!](#)

Code Smells Within Classes

- **Conditional Complexity**

- large conditional logic blocks, particularly blocks that tend to grow larger or change significantly over time.
- Consider alternative object-oriented approaches such as decorator, strategy, or state.

- **Combinatorial Explosion**

- Lots of code that does *almost* the same thing.. but with tiny variations in data or behavior.
- This can be difficult to refactor-- perhaps using generics or an interpreter?

- **Large Class**

- Large classes, like long methods, are difficult to read, understand, and troubleshoot.
- Large class can be restructured or broken into smaller

Code Smells Within Classes

- **Uncommunicative Name**

- Does the name of the method succinctly describe what that method does?
Could you read the method's name to another developer and have them explain to you what it does?

- **Inconsistent Names**

- set of standard terminology and stick to it throughout your methods.

- **Dead Code**

- Ruthlessly delete code that isn't being used

- **Speculative Generality**

- Write code to solve today's problems, and worry about tomorrow's problems when they actually materialize.
- Everyone loses in the "what if.." school of design.

Code Smells Between Classes

- **Alternative Classes with Different Interfaces**

- If two classes are similar on the inside, but different on the outside, perhaps they can be modified to share a common interface.

- **Primitive Obsession**

- If data type is sufficiently complex, write a class to represent it.

- **Data Class**

- Avoid classes that passively store data.
- Classes should contain data *and* methods to operate on that data, too.

- **Data Clumps**

- If you always see the same data hanging around together, maybe it belongs together.
- Consider rolling the related data up into a larger class.

Code Smells Between Classes

- **Refused Bequest**
 - Inherit from a class but never use any of the inherited functionality
- **Inappropriate Intimacy**
 - Classes that spend too much time together, or classes that interface in inappropriate ways.
 - Classes should know as little as possible about each other
- **Indecent Exposure**
 - Classes that unnecessarily expose their internals.
 - Aggressively refactor classes to minimize their public surface.
 - You should have a compelling reason for every item you make public. If you don't, hide it.
- **Feature Envy**
 - Methods that make extensive use of another class may belong in another class.
 - Move the method to the class it is so envious

Code Smells between Classes

- **Lazy Class**

- Classes should pull their weight.
- If a class isn't doing enough to pay for itself, it should be collapsed or combined into another class.

- **Message Chains**

- Long sequences of method calls or temporary variables to get routine data.
- Intermediaries are dependencies in disguise.

- **Middle Man**

- If a class is delegating all its work., then cut out the middleman.
- Beware classes that are merely wrappers over other classes or existing functionality in the framework.

- **Divergent Change**

- If changes to a class touch completely different parts of the class, it may contain too much unrelated functionality.
- Isolate the parts that changed in another class.

Code Smells between Classes

- Shotgun Surgery
 - If a change in one class requires cascading changes in several related classes
- Parallel Inheritance Hierarchies
 - Every time you make a subclass of one class, you must also make a subclass of another.
 - Consider folding the hierarchy into a single class.
- Solution Sprawl
 - If it takes five classes to do anything useful, you might have solution sprawl.
 - Consider simplifying and consolidating your design.

Code Refactoring Tools!

- **Static Code Analysis** - ESLint / Prettier, SonarQube / SonarLint
- **Inbuilt IDE** - JetBrains Suite (WebStorm, IntelliJ IDEA, ReSharper), Eclipse, Visual Studio
- **AI Assisted Tools:** Github Copilot, Gemini Code Assist, Claude Co-work, Cursor

Can AI be trusted as a Code Refactoring tool?

highly capable but strictly supervised

Syntax Modernization & Translation

Extracting and Decoupling

Readability Improvements

Test Generation as a Refactoring Step

Architectural Context Loss

"Hallucinated" Library Methods

Performance Blind Spots

Semantic Nuance

Refactoring Practices in the Context of Modern Code Review: An Industrial Case Study at Xerox

Eman Abdullah AlOmar*, Hussein AlRubaye†, Mohamed Wiem Mkaouer*, Ali Ouni‡, Marc

*Rochester Institute of Technology, Rochester, NY, USA

†Xerox Corporation, Rochester, NY, USA

‡ETS Montreal, University of Quebec, Montreal, QC, Canada

§University of Michigan, Dearborn, MI, USA

eman.alomar@mail.rit.edu, hussein.alrubaye@xerox.com, mwmvse@rit.edu, ali.ouni@etsmtl.ca, mar

Semantic Code Refactoring for Abstract Data Types

SHANKARA PAILOOR, University of Texas, USA

YUEPENG WANG, Simon Fraser University, Canada

IŞIL DILLIG, University of Texas, USA

Modifications to the data representation of an abstract data type (ADT) can require significant semantic observation, this paper presents a new method to automate takes as input the original ADT implementation, a new data *ntation invariant* (relating the old and new data representations), nentation that is semantically equivalent to the original version. led inductive synthesis (CEGIS) but leverages three key ideas tasks. First, our approach reduces the underlying relational naming-by-example problems, one for each method in the ADT. ques, based on logical abduction, to deduce code snippets that it utilizes a notion of *partial equivalence* to make inductive

Industry Experiences with Large-Scale Refactoring

James Ivers

jivers@sei.cmu.edu

CMU Software Engineering Institute
Pittsburgh, PA, USA

Robert L. Nord

rn@sei.cmu.edu

CMU Software Engineering Institute
Pittsburgh, PA, USA

Ipek Ozkaya

ozkaya@sei.cmu.edu

CMU Software Engineering Institute
Pittsburgh, PA, USA

Chris Seifried

cgseifried@sei.cmu.edu

CMU Software Engineering Institute
Pittsburgh, PA, USA

Christopher

ctimperley

Carnegie Mell
Pittsburgh

ABSTRACT

Software refactoring plays an important role in software engineering. Developers often turn to refactoring when they want to restructure software to improve its quality without changing its external behavior. Small-scale (floss) refactoring is common in industry and is often performed by a single developer in short sessions, even though developers do much of this work manually instead of using refactoring tools. However, some refactoring efforts are much larger in scale, requiring entire teams and months or years of effort, and the role of tools in these efforts is not as well studied. In this paper, we report on a survey we conducted with developers to understand large-scale refactoring and its tool support needs. Our results from 107 industry developers demonstrate that projects commonly go through multiple large-scale refactorings, each of which requires considerable effort. Our study finds that developers

How Do Code Refactorings Affect Energy Usage?*

Cagri Sahin

University of Delaware
United States
cagri@udel.edu

Lori Pollock

University of Delaware
United States
pollock@udel.edu

James Clause

University of Delaware
United States
clause@udel.edu

ABSTRACT

Context: Code refactoring's benefits to understandability, maintainability and extensibility are well known enough that automated support for refactoring is now common in IDEs. However, the decision to apply such transformations is currently performed without regard to the impacts of the refactorings on energy consumption. This is primarily due to a lack of information and tools to provide such relevant information.

General Terms

Design, Experimentation, Performance, Measurement

Keywords

code refactoring, energy usage, empirical study

Recap...

- Designs can deteriorate over a period of time
- Refactoring can help in managing the deterioration of design
 - One small step at a time
 - Don't refactor and add functionality at the same time

Systems are fairly large nowadays. Systems evolve over time and complexity needs to be managed!

Of course it does not make sense to **manually refactor!!!**

As a Software Engineer, one of the most important aspect of your job is to make the life of other developers/programmers/testers easier!!!

Class Activity

- Rewrite the code by addressing as many code smells as possible.

```
// Function to calculate things
function calculate(p, t, d) {
  let r = 0;

  if (t === "Member") {
    if (d === true) {
      // 20% discount plus 5 dollars off
      r = (p * 0.8) - 5;
    } else {
      r = p * 0.8;
    }
  } else if (t === "Guest") {
    if (d === true) {
      r = p - 5;
    } else {
      r = p;
    }
  } else {
    r = p;
  }

  // Log the result
  console.log("The total price is: " + r);
  return r;
}

// Example usage
calculate(100, "Member", true);
```

Solution

```
/**
 * Calculates the final price based on user status and coupon eligibility.
 * @param {number} basePrice - The initial cost.
 * @param {string} userType - The category of the user (e.g., "Member").
 * @param {boolean} hasCoupon - Whether the user has a discount coupon.
 * @returns {number} The final calculated price.
 */
function calculateTotalPrice(basePrice, userType = "", hasCoupon = false) {
  let finalPrice = basePrice;
  const normalizedType = userType.toLowerCase();

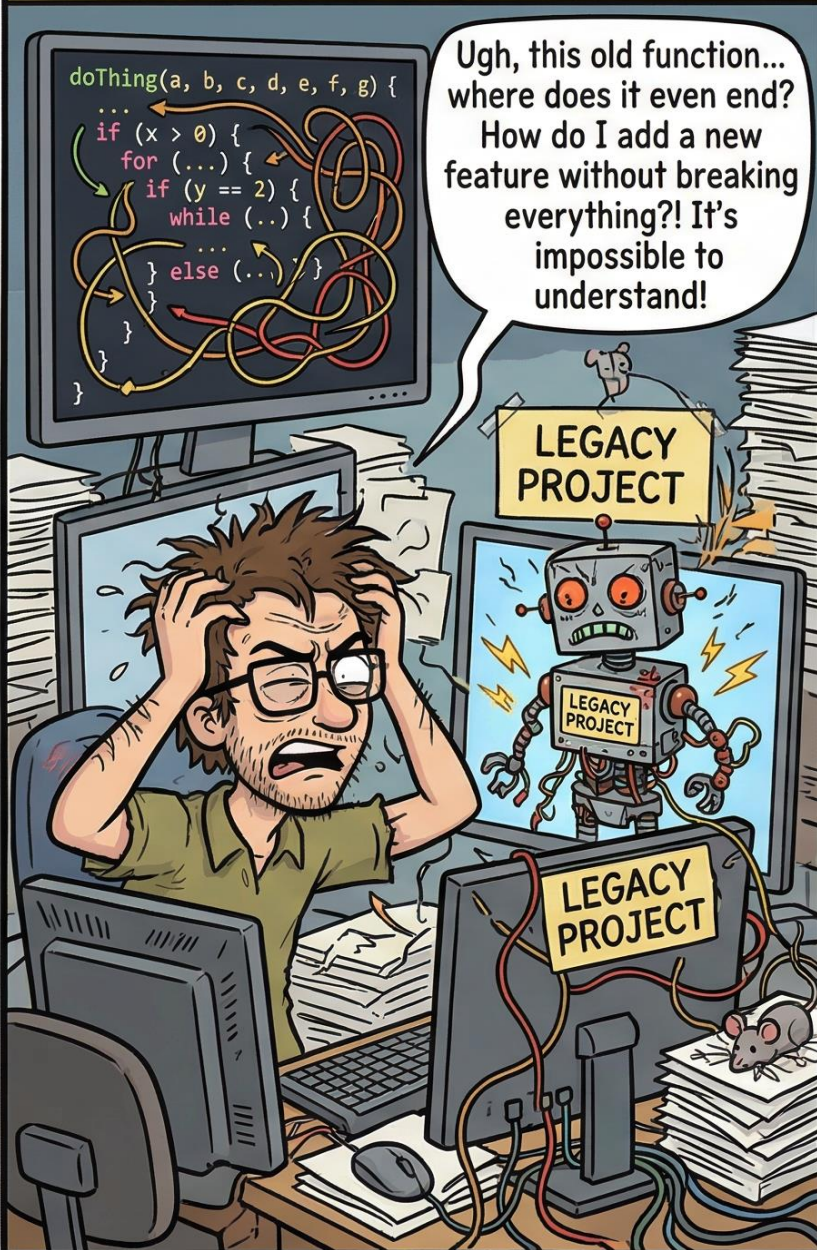
  // Apply membership discount
  if (normalizedType === "member") {
    finalPrice *= MEMBER_DISCOUNT_RATE;
  }

  // Apply flat coupon discount
  if (hasCoupon) {
    finalPrice -= COUPON_VALUE;
  }

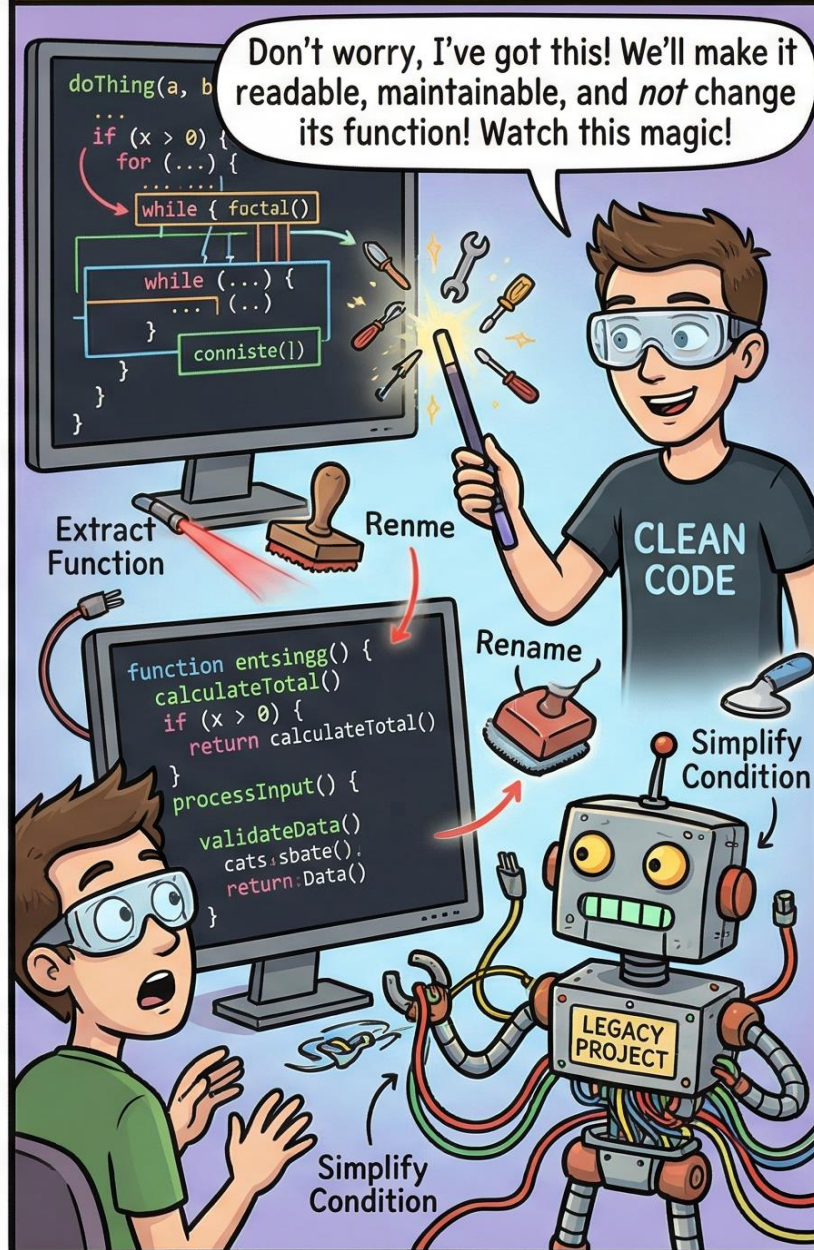
  // Ensure we don't return a negative price
  return Math.max(0, finalPrice);
}

// Example usage
const total = calculateTotalPrice(100, "Member", true);
console.log(`The total price is: ${total}`);
```


THE ENTANGLED LEGACY



ENTER THE REFACTORING WIZARD!



A BRIGHTER, MAINTAINABLE FUTURE

